Análisis predictivo de precios de alquiler de departamentos en Estados Unidos.

- **Grupo 10**
- **Integrantes:** Julian Domsch, Teo Poggi, Francisco Sbordi

DESCRIPCION DEL DATASET

Link del dataset: https://archive.ics.uci.edu/dataset/555/apartment+for+rent+classified

El dataset "Apartment for Rent Classifieds," contiene información sobre anuncios de alquiler de departamento recopilada de diferentes fuentes online. Cada registro en el dataset representa un anuncio de un departamento en alquiler en los Estados Unidos, y cada variable representa una caracteristica de este departamento.

OBJETIVO

Intentaremos predecir el precio de alquiler de los departamentos. Dado que nuestra variable objetivo, **price**, es una variable numérica continua, intentaremos buscar el modelo de regresion con mayor poder predictivo.

Evaluaremos nuestros modelos utilizando las métricas de **Raíz del Error Cuadrático Medio** (RMSE) y el **coeficiente de determinación** (R^2).

In [ ]:
```
#Cargar librerias
from matplotlib import pyplot as plt
import matplotlib.patches as mpatches
import plotly.express as px
from mpl_toolkits.mplot3d import Axes3D

import numpy as np
import pandas as pd
import seaborn as sns

from sklearn.model_selection import GridSearchCV, train_test_split, cross_val_score
from sklearn.linear_model import LinearRegression, LassoCV
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsRegressor
from sklearn.pipeline import Pipeline
```

In [ ]:
```
'''#Correr en colab
from google.colab import drive
import os
drive.mount('/content/drive')
rent = os.path.join('/content/drive/Shareddrives/TP_IAA/renta_100k.csv')
rentas = pd.read_csv(rent)
'''
#Correr Local
rentas = pd.read_csv('renta_100k.csv')
```

```
C:\Users\Julian\AppData\Local\Temp\ipykernel_832\1253174749.py:9: DtypeWarning: Colu
mns (15) have mixed types. Specify dtype option on import or set low_memory=False.
  rentas = pd.read_csv('renta_100k.csv')
```

EXPLORACION Y LIMPIEZA DEL DATASET

In [ ]:
```
rentas.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 99492 entries, 0 to 99491
Data columns (total 22 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   id             99492 non-null  int64
 1   category       99492 non-null  object
 2   title          99492 non-null  object
 3   body           99489 non-null  object
 4   amenities      83448 non-null  object
 5   bathrooms      99429 non-null  float64
 6   bedrooms       99368 non-null  float64
 7   currency       99492 non-null  object
 8   fee            99492 non-null  object
 9   has_photo      99492 non-null  object
 10  pets_allowed   39068 non-null  object
 11  price          99491 non-null  float64
 12  price_display  99491 non-null  object
 13  price_type     99492 non-null  object
 14  square_feet    99492 non-null  int64
 15  address        7943 non-null   object
 16  cityname       99190 non-null  object
 17  state          99190 non-null  object
 18  latitude       99467 non-null  float64
 19  longitude      99467 non-null  object
 20  source         99492 non-null  object
 21  time           99492 non-null  int64
dtypes: float64(4), int64(3), object(15)
memory usage: 16.7+ MB
```

El dataset original tiene 22 variables y 99492 datos. De las 22 variables 7 son numericas y 15 son categoricas.

Se observa que 11 de las variables contienen valores **NA**. Las variables "**address**" y "**pets_allowed**" tienen más de la mitad de sus datos faltantes, por lo cual decidimos

ignorarlas. La variable "amenities" también presenta una gran cantidad de datos faltantes, y la información en ella es poco clara, por lo que también será descartada. En el resto de las variables, eliminaremos los registros que contengan valores **NA**. Tambien arreglaremos el datatype de **longitude** y sus outliers.

In [ ]:
```
rentas['longitude'] = rentas['longitude'].astype(str).str.replace('.', '', regex=Fa
valores_debajo_neg1300 = (rentas['longitude'] < -1300).sum()
rentas = rentas[rentas['longitude'] >= -1300].reset_index(drop=True)
```

```
Cantidad de valores de longitude debajo de -1300: 89
```

Observamos la distribucion de los datos numericos

In [ ]:
```
print(rentas.describe())
```

```
                 id     bathrooms      bedrooms         price   square_feet  \
count  9.937800e+04  99316.000000  99254.000000  99377.000000  99378.000000
mean   5.358192e+09      1.445326      1.728021   1527.047657    956.524482
std    1.846305e+08      0.546859      0.748784    903.708595    417.454565
min    5.121046e+09      1.000000      0.000000    100.000000    101.000000
25%    5.197949e+09      1.000000      1.000000   1014.000000    729.000000
50%    5.508673e+09      1.000000      2.000000   1350.000000    900.000000
75%    5.509007e+09      2.000000      2.000000   1795.000000   1115.000000
max    5.669439e+09      9.000000      9.000000  52500.000000  50000.000000

           latitude     longitude          time
count  99378.000000  99378.000000  9.937800e+04
mean     369.386591   -915.144167  1.559658e+09
std       45.551546    157.194450  1.104487e+07
min      245.645000  -1242.265000  1.544174e+09
25%      337.465000  -1047.904000  1.550832e+09
50%      372.258000   -845.623000  1.568745e+09
75%      399.513000   -776.066000  1.568767e+09
max      488.467000   -687.788000  1.577391e+09
```

Podemos observar tambien que en **price** y **square_feet** que el valor maximo es varios ordenes de magnitud mas grande que el percentil 75% lo cual suponemos que se debe a la presencia de outliers.

In [ ]:
```
plt.figure(figsize=(16, 8))

# Primer subplot: Histograma de precios
plt.subplot(2, 2, 1)
sns.histplot(rentas['price'], kde=True)
plt.xlabel('Precio', fontsize=16)
plt.ylabel('Cantidad', fontsize=16)
plt.xticks(fontsize=14)
plt.yticks(fontsize=14)

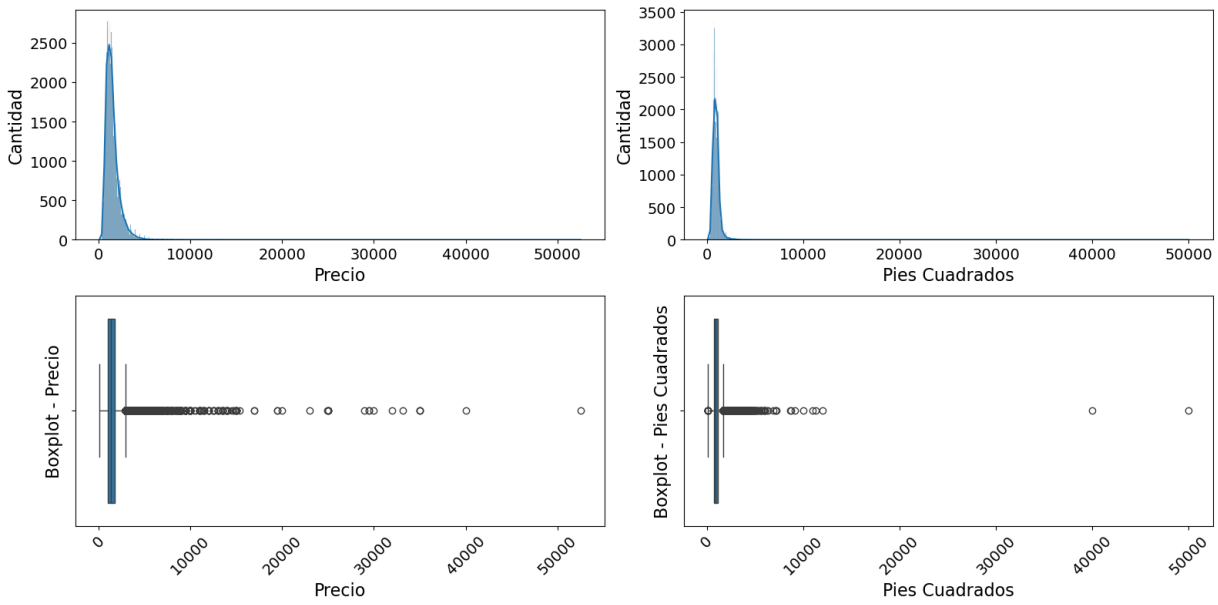
# Segundo subplot: Histograma de pies cuadrados
plt.subplot(2, 2, 2)
```

```
sns.histplot(rentas['square_feet'], kde=True)
plt.xlabel('Pies Cuadrados', fontsize=16)
plt.ylabel('Cantidad', fontsize=16)
plt.xticks(fontsize=14)
plt.yticks(fontsize=14)

# Tercer subplot: Boxplot de precios
plt.subplot(2, 2, 3)
sns.boxplot(x='price', data=rentas)
plt.xlabel('Precio', fontsize=16)
plt.ylabel('Boxplot - Precio', fontsize=16)
plt.xticks(rotation=45, fontsize=14)
plt.yticks(fontsize=14)

# Cuarto subplot: Boxplot de pies cuadrados
plt.subplot(2, 2, 4)
sns.boxplot(x='square_feet', data=rentas)
plt.xlabel('Pies Cuadrados', fontsize=16)
plt.ylabel('Boxplot - Pies Cuadrados', fontsize=16)
plt.xticks(rotation=45, fontsize=14)
plt.yticks(fontsize=14)

plt.tight_layout()
plt.show()
```



Con estos graficos podemos ver las distribucion de las variables price y square_feet donde confirmamos la precencia de outliers.

- Eliminamos valores duplicados

In [ ]:
```
# Elimino duplicados
rentas = rentas.drop_duplicates()
```

- Eliminamos variables con muchos datos faltantes, redundantes o no relevantes para el analisis.

In [ ]:
```
# Elimino variables no relevantes para el analisis
rentas = rentas.drop(columns=['id','category', 'address','price_display','title','b
```

- Limpiamos los outliers en la variable price y sqare_feet. Decidimos utilizar regla de los 3 sigmas, con esta tecnica eliminamos los outliers sin eliminar muchos datos.

In [ ]:
```
# Columnas a procesar
columns = ['price', 'square_feet']
# Filtrar *outliers* para cada columna usando la regla de las tres sigmas
for column in columns:
    data = rentas[column].dropna()
    mean = data.mean()
    std_dev = data.std()
    lower_bound = mean - 3 * std_dev
    upper_bound = mean + 3 * std_dev
    rentas = rentas[(rentas[column] >= lower_bound) & (rentas[column] <= upper_boun
```

- Limpiamos todos los datos que contengan NA.

In [ ]:
```
rentas.dropna(inplace=True)
```

In [ ]:
```
rentas.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 96748 entries, 0 to 99377
Data columns (total 11 columns):
 #   Column       Non-Null Count  Dtype  
---  ------       --------------  -----  
 0   bathrooms    96748 non-null  float64
 1   bedrooms     96748 non-null  float64
 2   fee          96748 non-null  object 
 3   has_photo    96748 non-null  object 
 4   price        96748 non-null  float64
 5   square_feet  96748 non-null  int64  
 6   cityname     96748 non-null  object 
 7   state        96748 non-null  object 
 8   latitude     96748 non-null  float64
 9   longitude    96748 non-null  float64
 10  source       96748 non-null  object 
dtypes: float64(5), int64(1), object(5)
memory usage: 8.9+ MB
```

Luego de la limpieza el dataset que utilizaremos para el analisis contiene 11 variables y 96748 datos. De los cuales 6 son numericos y 5 categoricos.

ANALISIS VISUAL DE LOS DATOS

In [ ]:
```
rentas_corr = rentas.drop(columns=['fee', 'has_photo', 'state', 'source', 'cityname
corr = rentas_corr.corr().round(2)
corr.style.format("{:.3f}").background_gradient(cmap='viridis')
```

Out[ ]:

	bathrooms	bedrooms	price	square_feet	latitude	longitude
bathrooms	1.000	0.650	0.270	0.730	-0.090	0.000
bedrooms	0.650	1.000	0.180	0.700	0.020	0.050
price	0.270	0.180	1.000	0.320	0.120	-0.090
square_feet	0.730	0.700	0.320	1.000	-0.030	0.110
latitude	-0.090	0.020	0.120	-0.030	1.000	0.090
longitude	0.000	0.050	-0.090	0.110	0.090	1.000

Se generó una matriz de correlación para analizar la relación de la variable **price** con las demás variables numéricas en el conjunto de datos. Los resultados muestran que las variables **square_feet**, **bathrooms** y **bedrooms** tienen la mayor correlación con **price**. Las variables **latitude** y **longitude** muestran una correlación muy baja con **price**, lo que indica que la ubicación geográfica, representada solo por coordenadas, tiene una influencia mínima en la variable **price**.

In [ ]:
```
# Línea de tendencia con ajuste lineal
z = np.polyfit(rentas['square_feet'], rentas['price'], 1)  # Cambiamos el orden de
p = np.poly1d(z)

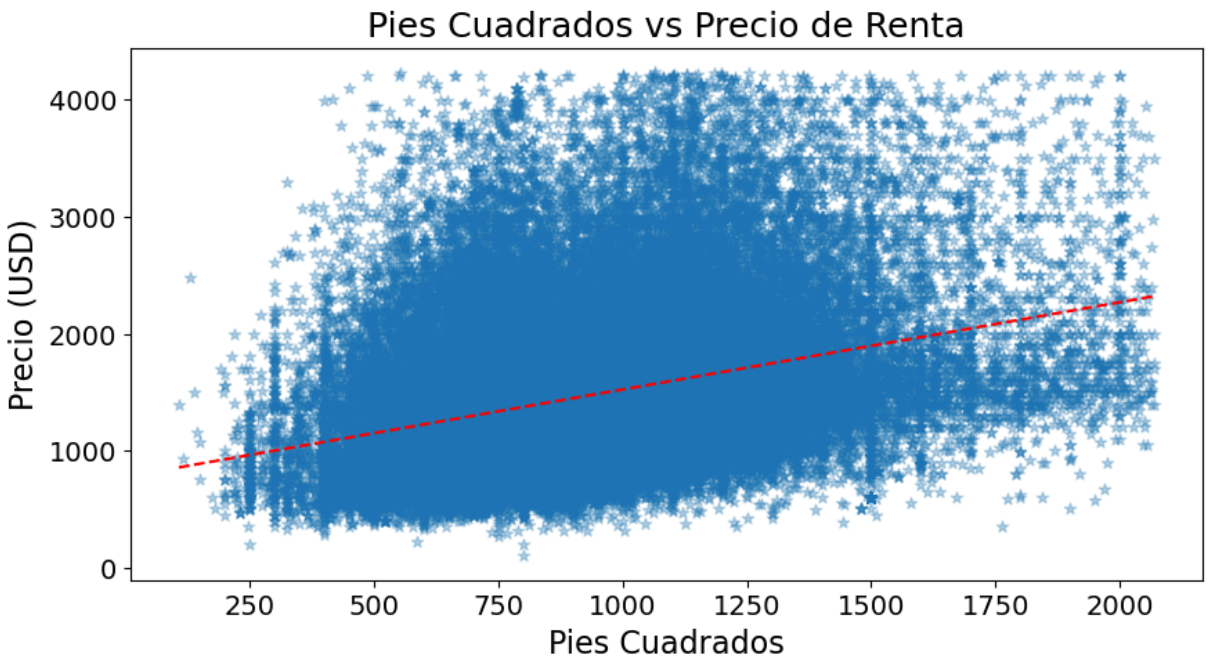
# Crear figura y gráfico de dispersión
fig, ax = plt.subplots(figsize=(10, 5))
mapeo = ax.scatter(rentas['square_feet'], rentas['price'], marker="*", alpha = 0.3,

# Rango de la línea de tendencia, usando los límites de 'square_feet'
x_line = np.linspace(rentas['square_feet'].min(), rentas['square_feet'].max(), 100)
ax.plot(x_line, p(x_line), "r--")

# Formato y etiquetas
ax.ticklabel_format(style='plain')
ax.set_title('Pies Cuadrados vs Precio de Renta', fontsize=18)
ax.set_xlabel('Pies Cuadrados', fontsize=16)
ax.set_ylabel('Precio (USD)', fontsize=16)
ax.tick_params(axis='x', labelsize=14)
ax.tick_params(axis='y', labelsize=14)

plt.show()
```

```
C:\Users\Julian\AppData\Local\Temp\ipykernel_832\3264381798.py:7: UserWarning: No da
ta for colormapping provided via 'c'. Parameters 'cmap' will be ignored
  mapeo = ax.scatter(rentas['square_feet'], rentas['price'], marker="*", alpha = 0.
3, cmap=plt.cm.viridis)
```



Se puede observar que hay una relacion creciente entre **square_feet** y **price**, sin embargo hay una gran dispercion de valores de **price** para un valor similar de **square_feet**.

In [ ]:
```
fig = plt.figure(figsize=(12, 8))
ax = fig.add_subplot(111, projection='3d')

# Variables para el gráfico
x = rentas['bathrooms']
y = rentas['bedrooms']
z = rentas['price']

# Crear el gráfico de dispersión en 3D
scatter = ax.scatter(x, y, z, alpha=0.3, marker="*")  #

# Calcular el ajuste del plano utilizando mínimos cuadrados
A = np.c_[x, y, np.ones(x.shape)]
C, _, _, _ = np.linalg.lstsq(A, z, rcond=None)

# Crear una cuadrícula para visualizar el plano
x_surf = np.linspace(x.min(), x.max(), 20)
y_surf = np.linspace(y.min(), y.max(), 20)
x_surf, y_surf = np.meshgrid(x_surf, y_surf)

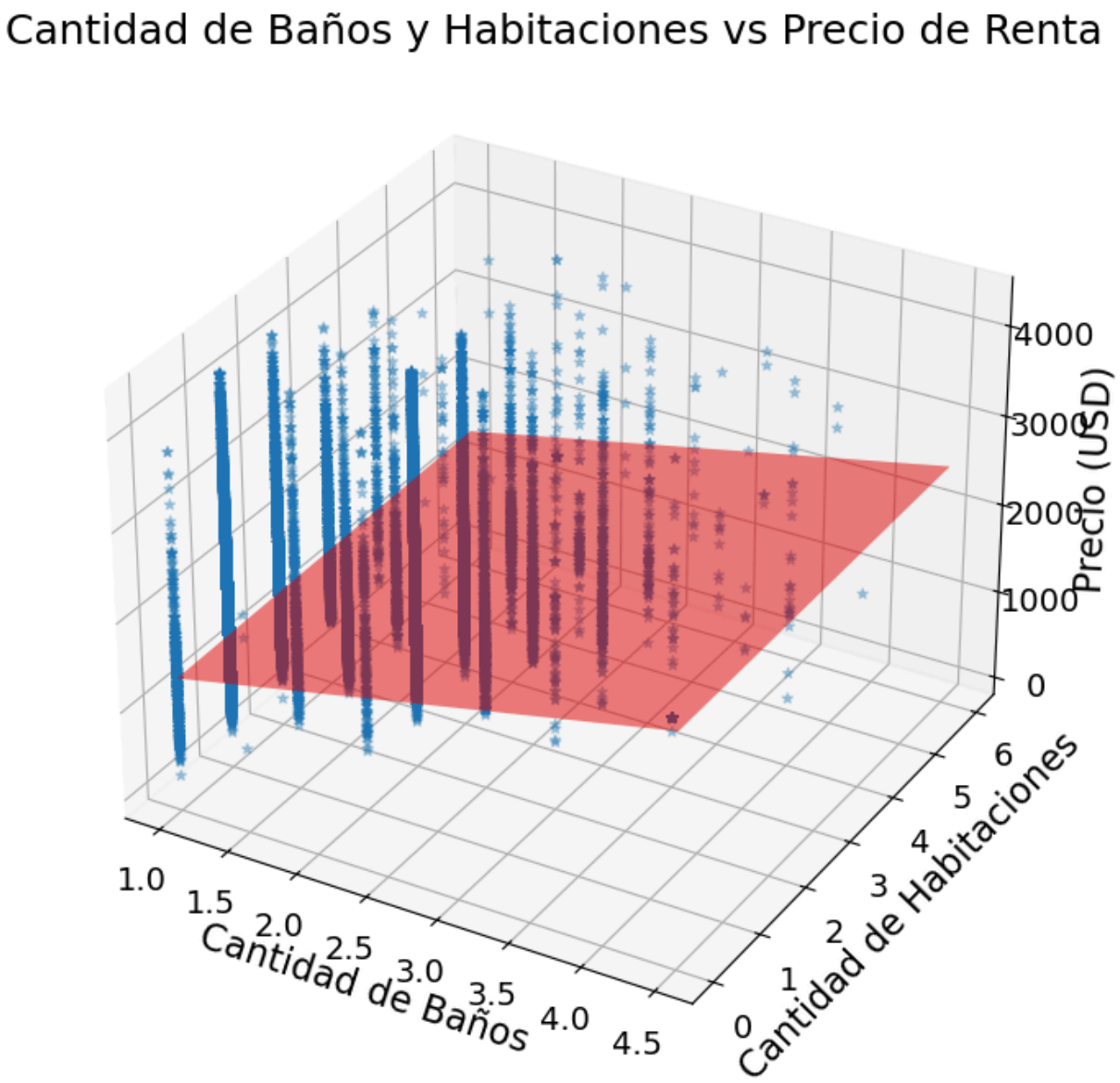
# Calcular los valores de z para la cuadrícula usando los coeficientes
z_surf = C[0] * x_surf + C[1] * y_surf + C[2]

# Agregar el plano de tendencia
ax.plot_surface(x_surf, y_surf, z_surf, color='red', alpha=0.5, rstride=100, cstrid

# Etiquetas y título
ax.set_title('Cantidad de Baños y Habitaciones vs Precio de Renta', fontsize=18)
ax.set_xlabel('Cantidad de Baños', fontsize=16)
ax.set_ylabel('Cantidad de Habitaciones', fontsize=16)
ax.set_zlabel('Precio (USD)', fontsize=16)
```

```
# Aumentar tamaño de los ticks
ax.tick_params(axis='x', labelsize=14)
ax.tick_params(axis='y', labelsize=14)
ax.tick_params(axis='z', labelsize=14)

plt.show()
```



Al igual que en **square_feet**, se observa una relación creciente entre **price** y las variables **bathrooms** y **bedrooms**. Sin embargo se observa una gran variabilidad en **price**.,

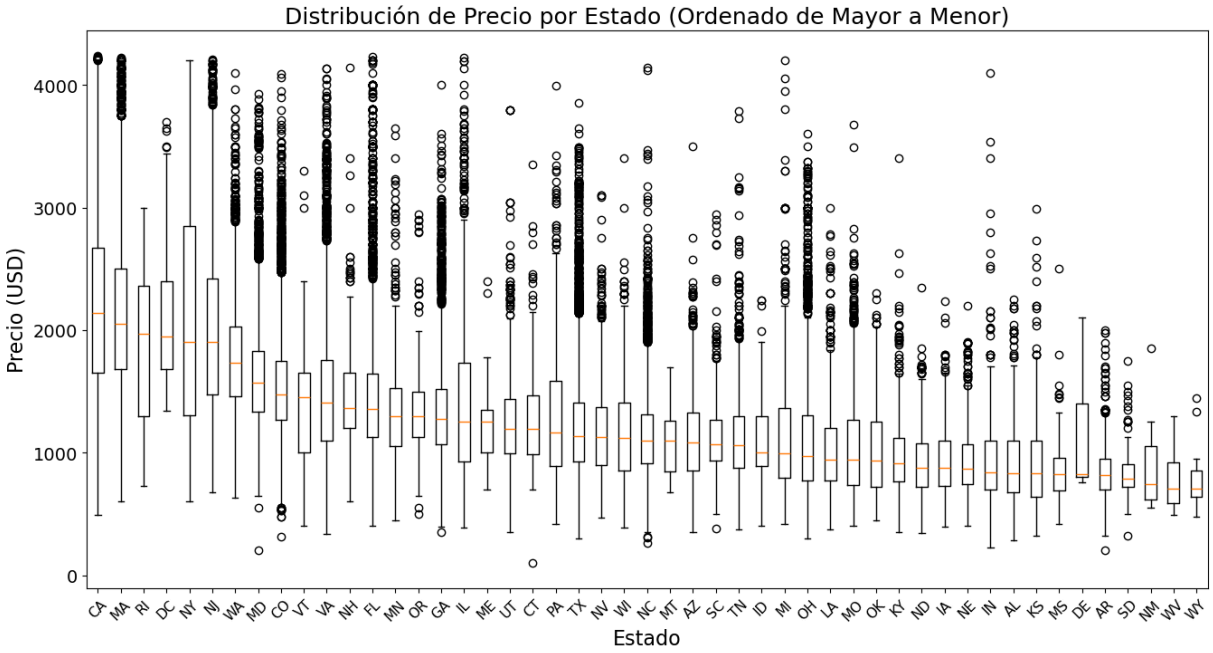
In [ ]:
```
# Calcular la mediana de price por estado y ordenar de mayor a menor
median_price_by_state = rentas.groupby('state')['price'].median().sort_values(ascen

# Obtener el orden de los estados y los datos ordenados
states_ordered = median_price_by_state.index
data_ordered = [rentas[rentas['state'] == state]['price'] for state in states_order

# Crear el boxplot
plt.figure(figsize=(16, 8))
```

```
plt.boxplot(data_ordered, labels=states_ordered)

# Etiquetas y título
plt.title('Distribución de Precio por Estado (Ordenado de Mayor a Menor)', fontsize
plt.xlabel('Estado', fontsize=16)
plt.ylabel('Precio (USD)', fontsize=16)
plt.xticks(rotation=45, fontsize=11)
plt.yticks(fontsize=14)
plt.show()
```



Aquí observamos cómo varía la distribución de **price** según cada valor de **state**, lo cual indica que el precio de los departamentos depende en cierta medida del estado o ciudad en el que se encuentran ubicados.

Entrenamiento y primero modelos del dataset

Regresion lineal simple

Entrenamiento de un modelo de regresion lineal utilizando las variables **'square_feet'** como atributo, y la variable **'price'** como target.

In [ ]:
```
# Preparar los datos
X = rentas[['square_feet']]
y = rentas['price']

# train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_sta
```

```
# Entrenamiento del modelo
model = LinearRegression()
model.fit(X_train, y_train)

# Predicciones
y_pred = model.predict(X_test)

# Evaluación del modelo (RMSE y R2)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)
print(f"RMSE en el conjunto de prueba: {rmse:.2f}")
print(f"R2 en el conjunto de prueba: {r2:.2f}")

# Benchmark del modelo
y_mean_pred = [y_train.mean()] * len(y_test)
rmse_benchmark = np.sqrt(mean_squared_error(y_test, y_mean_pred))
print(f"RMSE Benchmark (media): {rmse_benchmark:.2f}")

# Evaluación del modelo usando Cross-Validation
cv_scores = cross_val_score(model, X_train, y_train, cv=5, scoring='neg_mean_square
cv_r2_scores = cross_val_score(model, X_train, y_train, cv=5, scoring='r2')

# Convertir a RMSE y calcular la media de R2
rmse_cv_scores = np.sqrt(-cv_scores)

# Mostrar los resultados de la validación cruzada
print(f"RMSE (Cross-Validation): {rmse_cv_scores.mean():.2f} ± {rmse_cv_scores.std(
print(f"R2 (Cross-Validation): {cv_r2_scores.mean():.2f} ± {cv_r2_scores.std():.2f}
```

```
RMSE en el conjunto de prueba: 608.23
R2 en el conjunto de prueba: 0.11
RMSE Benchmark (media): 644.30
RMSE (Cross-Validation): 604.33 ± 4.50
R2 (Cross-Validation): 0.10 ± 0.01
```

El modelo de regresión lineal muestra un RMSE de 608.23 en el conjunto de prueba, mejorando ligeramente sobre el benchmark de la media (RMSE de 644.30). Sin embargo, con un r^2 de solo 0.11, el modelo explica poca variabilidad en los precios, lo que indica que la variable **square_feet** no alcanza para predecir **price**.

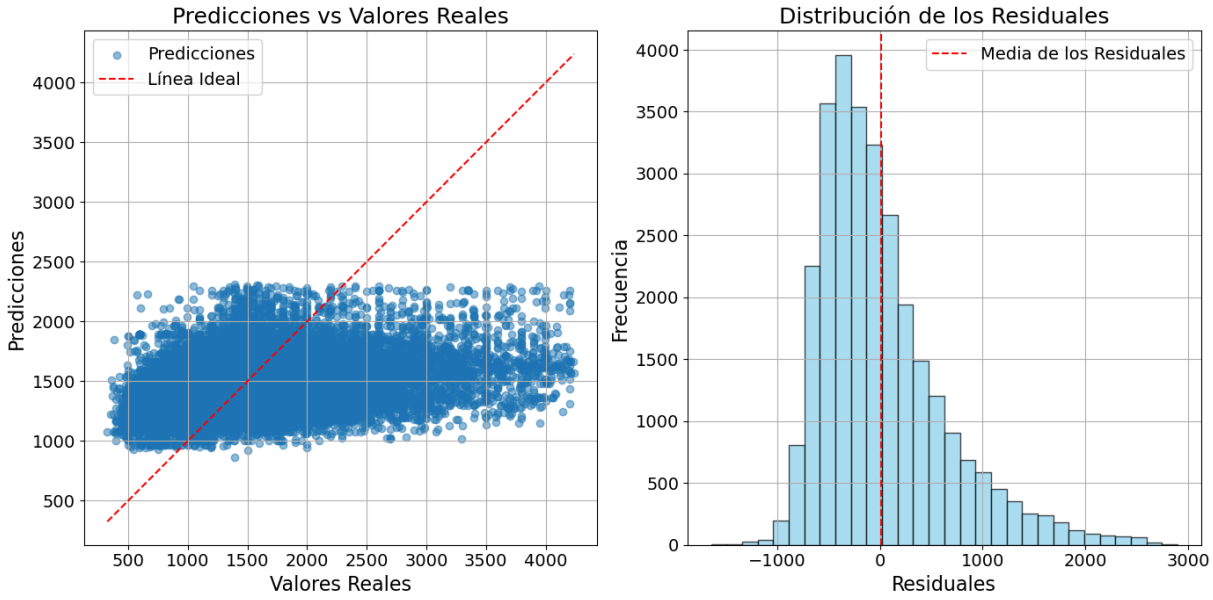
In [ ]:
```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 7))

# Gráfico de Predicciones vs Valores Reales
ax1.scatter(y_test, y_pred, alpha=0.5, label='Predicciones')
ax1.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', label='
ax1.set_xlabel('Valores Reales', fontsize=16)
ax1.set_ylabel('Predicciones', fontsize=16)
ax1.set_title('Predicciones vs Valores Reales', fontsize=18)
ax1.legend(fontsize=14)
ax1.grid(True)
ax1.tick_params(axis='x', labelsize=14)
ax1.tick_params(axis='y', labelsize=14)

# Gráfico de Distribución de los Residuales
residuals = y_test - y_pred
```

```
ax2.hist(residuals, bins=30, alpha=0.7, color='skyblue', edgecolor='black')
ax2.axvline(residuals.mean(), color='r', linestyle='--', label='Media de los Residu
ax2.set_xlabel('Residuales', fontsize=16)
ax2.set_ylabel('Frecuencia', fontsize=16)
ax2.set_title('Distribución de los Residuales', fontsize=18)
ax2.legend(fontsize=14)
ax2.grid(True)
ax2.tick_params(axis='x', labelsize=14)
ax2.tick_params(axis='y', labelsize=14)

plt.tight_layout()
plt.show()
```



Los puntos dispersos muestran que las predicciones no estan siguiendo una relacion lineal con los valores reales. La mayoria de los puntos quedan por debajo de la linea de tendencia, lo que muestra una subestimacion de los valores reales.

Regresion lineal multiple

Entrenamiento de un modelo de regresion lineal utilizando las variables **'state'** (a traves de hot encoding), **'square_feet'**, **'bathrooms'** y **'bedrooms'** como atributos, y la variable **'price'** como target.

In [ ]:
```
# Selección de atributos y target
X = rentas[['square_feet', 'bathrooms', 'bedrooms', 'longitude', 'latitude']]
y = rentas['price']

# train test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_sta

# Escalado
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)

# Entrenamiento del modelo
model = LinearRegression()
model.fit(X_train_scaled, y_train)

# Predicciones
y_pred = model.predict(X_test_scaled)

# Evaluación del modelo (RMSE y R2)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)
print(f"RMSE en el conjunto de prueba: {rmse:.2f}")
print(f"R2 en el conjunto de prueba: {r2:.2f}")

# Benchmark
y_mean_pred = [y_train.mean()] * len(y_test)
rmse_benchmark = np.sqrt(mean_squared_error(y_test, y_mean_pred))
print(f"RMSE Benchmark (media): {rmse_benchmark:.2f}")

# Validación cruzada
cv_rmse_scores = np.sqrt(-cross_val_score(model, X_train_scaled, y_train, cv=5, sco
cv_r2_scores = cross_val_score(model, X_train_scaled, y_train, cv=5, scoring='r2')

# Resultados de la validación cruzada
cv_results = pd.DataFrame({
    'Fold': range(1, len(cv_rmse_scores) + 1),
    'RMSE': cv_rmse_scores,
    'R2': cv_r2_scores
})

print(cv_results)

# Media y desviación estándar de RMSE y R2
mean_rmse = cv_rmse_scores.mean()
std_rmse = cv_rmse_scores.std()
mean_r2 = cv_r2_scores.mean()
std_r2 = cv_r2_scores.std()

print(f"RMSE (Cross-Validation): {mean_rmse:.2f} ± {std_rmse:.2f}")
print(f"R2 (Cross-Validation): {mean_r2:.2f} ± {std_r2:.2f}")
```

```
RMSE en el conjunto de prueba: 593.19
R2 en el conjunto de prueba: 0.15
RMSE Benchmark (media): 644.30
   Fold        RMSE        R2
0     1  584.275656  0.146402
1     2  597.259908  0.146033
2     3  586.039010  0.151315
3     4  584.183143  0.146768
4     5  590.405353  0.160737
RMSE (Cross-Validation): 588.43 ± 4.96
R2 (Cross-Validation): 0.15 ± 0.01
```

Podemos observar que el RMSE en el conjunto de prueba es significativamente menor que el del modelo anterior, lo que indica que las variables 'bathrooms', 'bedrooms' y 'region,'

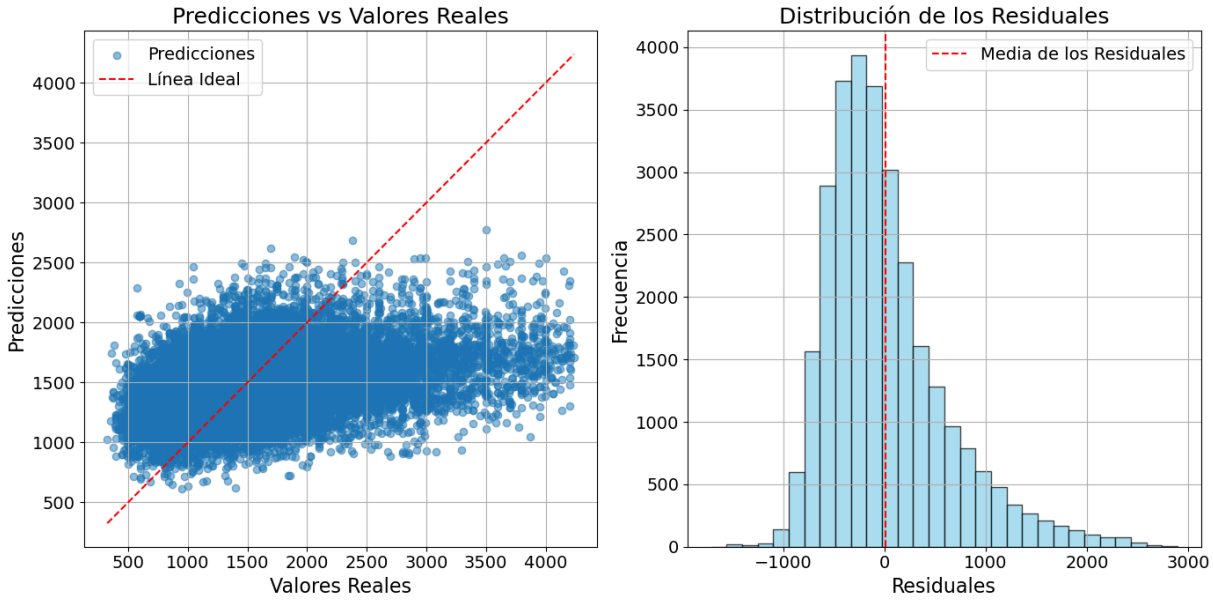
aportan a una mayor prediccion del modelo. Además, la variabilidad en los resultados de la validación cruzada tambien fue menor, lo que sugiere que este modelo es más consistente y no parece estar sobreajustado.

In [ ]:
```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 7))

# Gráfico de Predicciones vs Valores Reales
ax1.scatter(y_test, y_pred, alpha=0.5, label='Predicciones')
ax1.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', label='
ax1.set_xlabel('Valores Reales', fontsize=16)
ax1.set_ylabel('Predicciones', fontsize=16)
ax1.set_title('Predicciones vs Valores Reales', fontsize=18)
ax1.legend(fontsize=14)
ax1.grid(True)
ax1.tick_params(axis='x', labelsize=14)
ax1.tick_params(axis='y', labelsize=14)

# Gráfico de Distribución de los Residuales
residuals = y_test - y_pred
ax2.hist(residuals, bins=30, alpha=0.7, color='skyblue', edgecolor='black')
ax2.axvline(residuals.mean(), color='r', linestyle='--', label='Media de los Residu
ax2.set_xlabel('Residuales', fontsize=16)
ax2.set_ylabel('Frecuencia', fontsize=16)
ax2.set_title('Distribución de los Residuales', fontsize=18)
ax2.legend(fontsize=14)
ax2.grid(True)
ax2.tick_params(axis='x', labelsize=14)
ax2.tick_params(axis='y', labelsize=14)

plt.tight_layout()
plt.show()
```



Modelo LassoCV

Utilizamos Lasso para ver como impactan las distintas variables numericas y los valores de las variables categoricas en el precio.

In [ ]:
```
# Selección de atributos y target
X = pd.get_dummies(rentas[['square_feet', 'bathrooms', 'bedrooms', 'state', 'cityna
y = rentas['price']

# División del conjunto de datos en entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_sta

# Escalado de datos
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Inicializar y entrenar el modelo LassoCV (optimización de alpha)
lasso_cv = LassoCV(cv=5, random_state=42)
lasso_cv.fit(X_train_scaled, y_train)

# Mejor alpha encontrado por LassoCV
best_alpha = lasso_cv.alpha_
print(f"Mejor valor de alpha (LassoCV): {best_alpha:.3f}")

# Predicciones
y_pred_lasso = lasso_cv.predict(X_test_scaled)

# Evaluar el modelo
rmse_lasso = np.sqrt(mean_squared_error(y_test, y_pred_lasso))
r2_lasso = r2_score(y_test, y_pred_lasso)
print(f"RMSE del modelo Lasso: {rmse_lasso:.2f}")
print(f"R2 del modelo Lasso: {r2_lasso:.2f}")
print(f"--------------------------")
# Mostrar los coeficientes del modelo LassoCV
coef_lasso = lasso_cv.coef_

# Ordenar los coeficientes por su valor absoluto
coef_sorted = sorted(zip(coef_lasso, X.columns), key=lambda x: abs(x[0]), reverse=T

# Imprimir solo los 25 primeros coeficientes ordenados
print("Coeficientes del modelo LassoCV (ordenados por valor absoluto):")
for coef, feature in coef_sorted[:25]:
    print(f"{feature}: {coef:.4f}")
```

```
Mejor valor de alpha (LassoCV): 0.262
RMSE del modelo Lasso: 326.19
R2 del modelo Lasso: 0.74
--------------------------
Coeficientes del modelo LassoCV (ordenados por valor absoluto):
state_CA: 335.1039
square_feet: 222.4799
state_VA: 205.6888
state_MA: 200.9628
state_NJ: 166.9249
state_WA: 116.7297
state_CO: 101.9016
state_MD: 99.1965
bathrooms: 83.5596
cityname_Los Angeles: 70.8631
cityname_Atlanta: 68.4715
cityname_Boston: 63.3995
cityname_Jersey City: 63.3608
cityname_Dallas: 60.8181
state_NH: 59.2825
cityname_New York: 55.0912
cityname_Chicago: 51.8111
cityname_Newport News: -49.6620
cityname_Culver City: 48.3387
state_DC: 46.9023
bedrooms: -45.6389
cityname_Cambridge: 44.9855
state_NY: 44.1334
cityname_Norfolk: -43.7324
cityname_Richmond: -43.4430
```

Con un valor óptimo de alpha de 0.262, logramos un RMSE de 326.19, que representa una notable reducción frente al modelo anterior, y un coeficiente de determinación (R^2) del 74%. Además, observamos que variables geográficas, como el estado y la ciudad donde se ubica la vivienda, pueden ser tan relevantes como el tamaño de la propiedad.

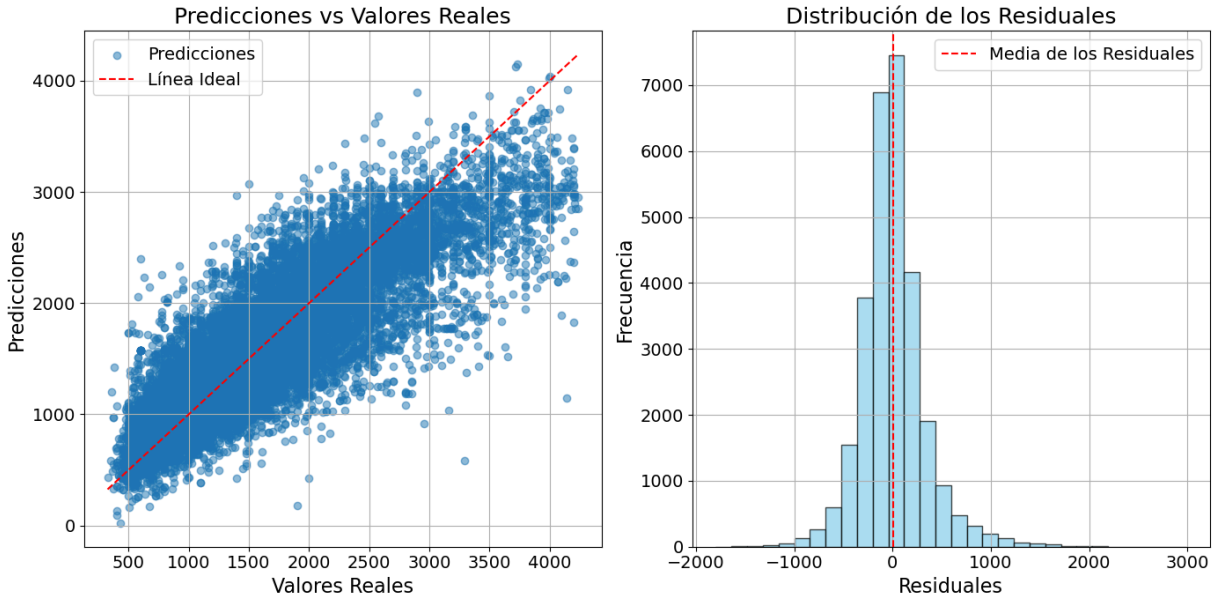
In [ ]:
```python
# Crear la figura y los ejes para los dos gráficos
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 7))

# Gráfico de Predicciones vs Valores Reales
ax1.scatter(y_test, y_pred_lasso, alpha=0.5, label='Predicciones')
ax1.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', label='
ax1.set_xlabel('Valores Reales', fontsize=16)
ax1.set_ylabel('Predicciones', fontsize=16)
ax1.set_title('Predicciones vs Valores Reales', fontsize=18)
ax1.legend(fontsize=14)
ax1.grid(True)
ax1.tick_params(axis='x', labelsize=14)
ax1.tick_params(axis='y', labelsize=14)

# Gráfico de Distribución de los Residuales
residuals = y_test - y_pred_lasso
ax2.hist(residuals, bins=30, alpha=0.7, color='skyblue', edgecolor='black')
ax2.axvline(residuals.mean(), color='r', linestyle='--', label='Media de los Residu
ax2.set_xlabel('Residuales', fontsize=16)
```

```
ax2.set_ylabel('Frecuencia', fontsize=16)
ax2.set_title('Distribución de los Residuales', fontsize=18)
ax2.legend(fontsize=14)
ax2.grid(True)
ax2.tick_params(axis='x', labelsize=14)
ax2.tick_params(axis='y', labelsize=14)

# Ajustar el layout y mostrar los gráficos
plt.tight_layout()
plt.show()
```



Modelo KNN-Regresion con optimizacion de hiperparametros

In [ ]:

```
# Cargar y preparar los datos
X = pd.get_dummies(rentas[['square_feet', 'bathrooms', 'bedrooms', 'state','citynam
y = rentas['price']

# División de datos en entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_sta

# Definir pipeline que incluya el escalado y el modelo KNeighborsRegressor
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('knn', KNeighborsRegressor())
])

# Configurar los parámetros para GridSearchCV
param_grid = {'knn__n_neighbors': range(1, 10)}
grid_search = GridSearchCV(estimator=pipeline,
                           param_grid=param_grid,
                           scoring='neg_mean_squared_error',
                           cv=5,
                           n_jobs=8)
```

```python
# Entrenar el GridSearchCV
grid_search.fit(X_train, y_train)

# Obtener el mejor valor de n_neighbors
best_n_neighbors = grid_search.best_params_['knn__n_neighbors']
print(f"Mejor valor de n_neighbors: {best_n_neighbors}")

# Entrenar el mejor modelo encontrado por GridSearchCV
best_knn_model = grid_search.best_estimator_
best_knn_model.fit(X_train, y_train)

# Predicciones en el conjunto de prueba
y_pred_knn = best_knn_model.predict(X_test)

# Evaluación del modelo en el conjunto de prueba
rmse_knn = np.sqrt(mean_squared_error(y_test, y_pred_knn))
r2_knn = r2_score(y_test, y_pred_knn)
print(f"RMSE en el conjunto de prueba: {rmse_knn:.2f}")
print(f"R² en el conjunto de prueba: {r2_knn:.2f}")

# Benchmark usando la media de entrenamiento como predicción
y_mean_pred = [y_train.mean()] * len(y_test)
rmse_benchmark = np.sqrt(mean_squared_error(y_test, y_mean_pred))
print(f"RMSE Benchmark (media): {rmse_benchmark:.2f}")

# Validación cruzada en el mejor modelo
cv_scores = cross_val_score(best_knn_model, X_train, y_train, cv=5, scoring='neg_me
rmse_cv_scores = np.sqrt(-cv_scores)

# Resultados de la validación cruzada
cv_results = pd.DataFrame({
    'Fold': range(1, len(rmse_cv_scores) + 1),
    'RMSE': rmse_cv_scores
})

print("\nResultados de la validación cruzada:")
print(cv_results)

# Media y desviación estándar de RMSE en validación cruzada
mean_rmse_cv = rmse_cv_scores.mean()
std_rmse_cv = rmse_cv_scores.std()
print(f"\nRMSE (Cross-Validation): {mean_rmse_cv:.2f} ± {std_rmse_cv:.2f}")

# Calcular R² para la validación cruzada
cv_r2_scores = cross_val_score(best_knn_model, X_train, y_train, cv=5, scoring='r2'
mean_r2_cv = cv_r2_scores.mean()
std_r2_cv = cv_r2_scores.std()
print(f"\nR² (Cross-Validation): {mean_r2_cv:.2f} ± {std_r2_cv:.2f}")
```

```
Mejor valor de n_neighbors: 4
RMSE en el conjunto de prueba: 302.27
R² en el conjunto de prueba: 0.78
RMSE Benchmark (media): 644.30

Resultados de la validación cruzada:
   Fold        RMSE
0     1  310.104684
1     2  315.076483
2     3  308.164623
3     4  316.128402
4     5  317.665870

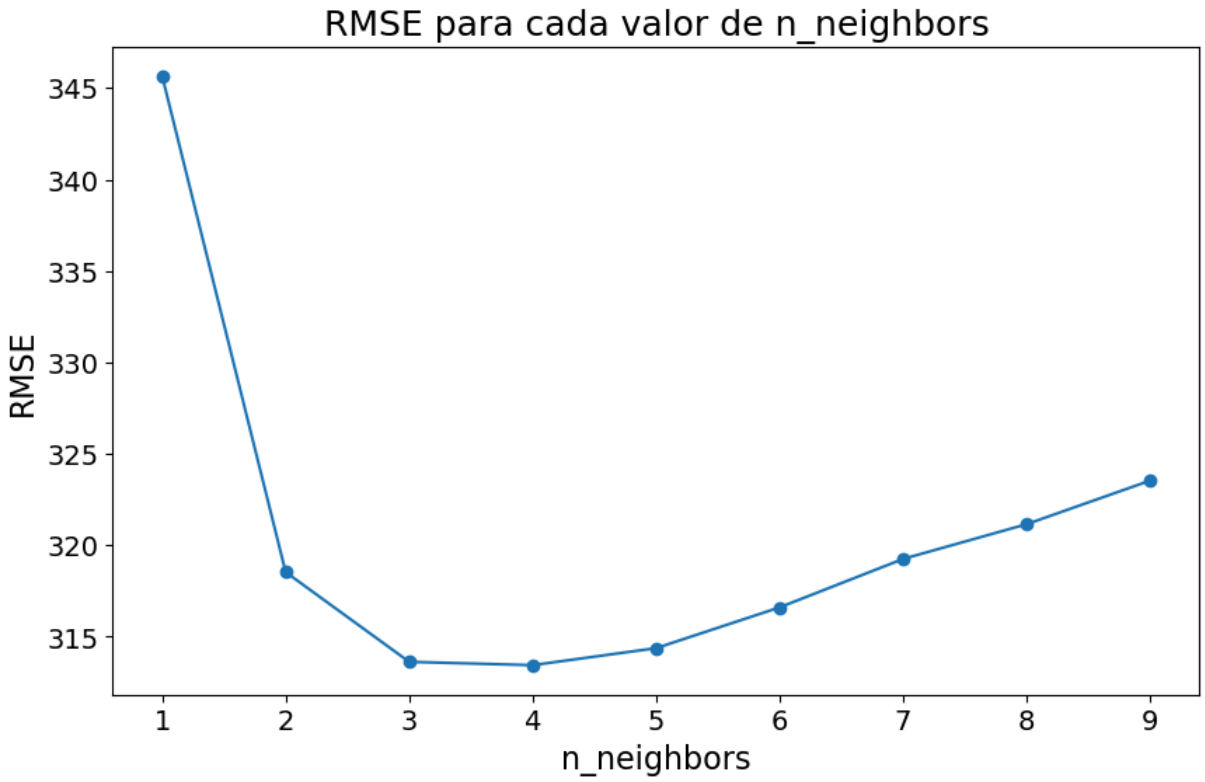
RMSE (Cross-Validation): 313.43 ± 3.65

R² (Cross-Validation): 0.76 ± 0.01
```

Out[ ]:
```
"\n# Obtener y graficar los resultados de cada valor de n_neighbors en GridSearchC
V\nresults = grid_search.cv_results_\nneighbors = results['param_knn__n_neighbor
s'].data\nmean_mse_scores = results['mean_test_score']\nrmse_scores = np.sqrt(-mea
n_mse_scores)  # Convertir MSE negativo a RMSE positivo\n\n# Visualizar RMSE para
cada valor de n_neighbors\nplt.figure(figsize=(10, 6))\nplt.plot(neighbors, rmse_s
cores, marker='o', linestyle='-')\nplt.xlabel('n_neighbors', fontsize=16)\nplt.yla
bel('RMSE', fontsize=16)\nplt.title('RMSE para cada valor de n_neighbors', fontsiz
e=18)\nplt.tick_params(axis='x', labelsize=14)\nplt.tick_params(axis='y', labelsiz
e=14)\nplt.show()"
```

In [ ]:
```python
# Obtener y graficar los resultados de cada valor de n_neighbors en GridSearchCV
results = grid_search.cv_results_
neighbors = results['param_knn__n_neighbors'].data
mean_mse_scores = results['mean_test_score']
rmse_scores = np.sqrt(-mean_mse_scores)

# Visualizar RMSE para cada valor de n_neighbors
plt.figure(figsize=(10, 6))
plt.plot(neighbors, rmse_scores, marker='o', linestyle='-')
plt.xlabel('n_neighbors', fontsize=16)
plt.ylabel('RMSE', fontsize=16)
plt.title('RMSE para cada valor de n_neighbors', fontsize=18)
plt.tick_params(axis='x', labelsize=14)
plt.tick_params(axis='y', labelsize=14)
plt.show()
```



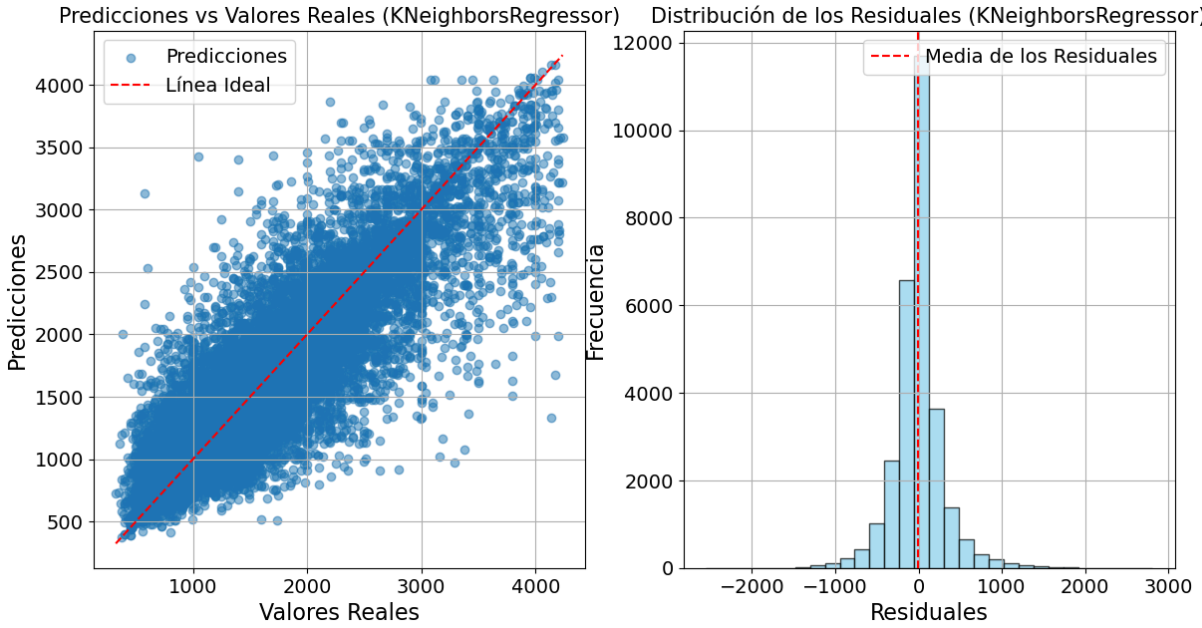
In [ ]:

```python
# Crear la figura y los ejes para los dos gráficos
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 7))

# Gráfico de Predicciones vs Valores Reales para el modelo KNeighborsRegressor
ax1.scatter(y_test, y_pred_knn, alpha=0.5, label='Predicciones')
ax1.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', label='
ax1.set_xlabel('Valores Reales', fontsize=16)
ax1.set_ylabel('Predicciones', fontsize=16)
ax1.set_title('Predicciones vs Valores Reales (KNeighborsRegressor)', fontsize=15)
ax1.legend(fontsize=14)
ax1.grid(True)
ax1.tick_params(axis='x', labelsize=14)
ax1.tick_params(axis='y', labelsize=14)

# Gráfico de Distribución de los Residuales para el modelo KNeighborsRegressor
residuals = y_test - y_pred_knn
ax2.hist(residuals, bins=30, alpha=0.7, color='skyblue', edgecolor='black')
ax2.axvline(residuals.mean(), color='r', linestyle='--', label='Media de los Residu
ax2.set_xlabel('Residuales', fontsize=16)
ax2.set_ylabel('Frecuencia', fontsize=16)
ax2.set_title('Distribución de los Residuales (KNeighborsRegressor)', fontsize=15)
ax2.legend(fontsize=14)
ax2.grid(True)
ax2.tick_params(axis='x', labelsize=14)
ax2.tick_params(axis='y', labelsize=14)

# Ajustar el layout y mostrar los gráficos
plt.show()
```



El modelo con el mejor valor de n_neighbors igual a 4 logró un RMSE de 302.27 y un R^2 de 0.78 en el conjunto de prueba, lo que supera al modelo Lasso y la comparación con el benchmark de la media muestra que el modelo KNN tiene un desempeño significativamente superior a una predicción básica. En la gráfica de la distribución de los residuales, vemos una distribución simétrica centrada en cero, lo cual nos muestra que el modelo no presenta un sesgo importante. La validacion cruzada tiene un promedio cercano al RMSE y una desviacion estandar de 3.65 lo que indica que es constante. Ademas tiene un R^2 de 0,76 siendo el modelo mas predictivo de los 4 realizados. Todavia se nota gran dispercion de los puntos en el grafico, sobre todo en los valores mas altos, lo cual muestra que el modelo sigue teniendo limitaciones.

Como conclusion elegimos el modelo KNeighborsRegressor como nuesto modelo favorito para la prediccion de precios, ya que de los modelos probados es el que tiene mayor poder predictivo y mayor desempeño. Además, resultó ser menos costoso computacionalmente en comparación con el modelo LassoCV, lo que lo hace una opción más eficiente en términos de recursos y tiempo de procesamiento.

In [ ]:
```
#Fin chequeo de impresion
```